

LAPORAN AKHIR
SISTEM INVENTARIS BARANG KANTOR



Oleh:

Najwa Nathania	2510631250051
Regina Martha Secunda	2510631250074
Sheva Ardian Zein	2510631250047

PRODI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
KARAWANG

2026

BAB I

PENDAHULUAN

A. Latar Belakang

Inventarisasi barang di kantor merupakan kegiatan yang sangat penting untuk mendukung kelancaran operasional suatu Perusahaan atau instansi. Barang-barang seperti perangkat elektronik, alat tulis kantor (ATK), maupun furniture memiliki peran penting dalam menunjang aktivitas kerja sehari-hari. Oleh karena itu, diperlukan pengelolaan inventaris yang baik agar proses pencatatan, penyimpanan, dan pemantauan barang dapat dilakukan secara teratur dan efisien. Namun, tanpa adanya sistem pencatatan yang terstruktur, sering muncul berbagai permasalahan seperti kesulitan dalam menemukan barang, risiko kehilangan atau kerusakan barang, kurangnya transparansi terhadap kondisi inventaris, serta pengambilan keputusan yang kurang akurat terkait kebutuhan pengadaan maupun perawatan barang. Kondisi tersebut dapat menurunkan efisiensi kerja karena proses pencarian dan pengelolaan inventaris menjadi lebih lambat dan kurang sistematis.

Pada kenyataannya, proses pengelolaan inventaris di beberapa tempat masih dilakukan secara manual. Cara tersebut sering menimbulkan berbagai permasalahan, seperti kesalahan pencatatan data, data yang kurang terorganisir, serta kurang efektifnya proses pemantauan kondisi dan jumlah inventaris. Selain itu, proses pencarian dan pengurutan data juga memerlukan waktu yang lebih lama, terutama ketika jumlah barang yang dikelola semakin banyak. Untuk mengatasi permasalahan tersebut, diperlukan sebuah sistem inventaris sederhana yang mampu memenuhi kebutuhan dasar pengelolaan barang secara lebih efektif dan terstruktur. Oleh karena itu, dibuatlah **Sistem Inventaris Kantor** menggunakan bahasa pemrograman Java sebagai solusi praktis dalam pengelolaan data inventaris. Program ini menyediakan fitur CRUD (Create, Read, Update, Delete), pencarian data, pengurutan data, pemulihan data (*restore*), serta statistik kondisi barang.

Dalam implementasinya, sistem ini menerapkan konsep Object Oriented Programming (OOP), struktur data *array*, serta algoritma *searching* dan *sorting*. Dengan adanya sistem ini, proses inventarisasi diharapkan menjadi lebih terstruktur, efisien, transparan, dan fleksibel. Selain itu, program menjadi sarana pembelajaran dalam

mengimplementasikan konsep pemrograman, struktur data, dan algoritma pada studi kasus nyata.

B. Tujuan

Berdasarkan permasalahan yang telah dijelaskan pada latar belakang, tujuan dari pembuatan Sistem Inventaris Kantor adalah sebagai berikut:

1. Membantu proses pengelolaan data inventaris agar rapi, terstruktur, dan mudah diakses.
2. Mempermudah pengguna dalam melakukan pencatatan, pencarian, serta pengurutan data barang secara lebih cepat dan efisien.
3. Mengurangi Risiko kesalahan pencatatan dan kehilangan data yang sering terjadi pada pengelolaan inventaris secara manual.
4. Menyediakan fitur pengelolaan inventaris seperti tambah data, edit data, hapus data, restore data, dan statistik barang dalam satu sistem terintegrasi.
5. Mengimplementasikan konsep pemrograman berorientasi objek (OOP), struktur data array, serta algoritma *searching* dan *sorting* pada bahasa pemrograman Java.
6. Menjadi sarana pembelajaran dalam menerapkan struktur data dan algoritma ke dalam studi kasus pengelolaan inventaris kantor.

BAB II

ANALISIS KEBUTUHAN

A. Analisis Kebutuhan Sistem

Sistem Inventaris Kantor dikembangkan untuk mendukung pengelolaan data barang agar lebih terstruktur, efisien, dan mudah digunakan. Sistem ini dibuat menggunakan bahasa pemrograman **Java** dengan menerapkan konsep **Object Oriented Programming (OOP)**, pemanfaatan struktur data array, serta penerapan algoritma *searching* dan *sorting*. Melalui sistem ini, pengguna dapat melakukan pencatatan inventaris secara sistematis, mulai dari penambahan, pengeditan, penghapusan, hingga pemulihan data barang. Fitur pencarian dan pengurutan membantu pengguna menemukan informasi dengan cepat dan akurat, sementara mekanisme *soft delete* memberikan fleksibilitas karena data yang dihapus masih dapat dipulihkan kembali melalui fungsi *restore*. Secara keseluruhan, sistem ini tidak hanya meningkatkan keteraturan pencatatan inventaris, tetapi juga mempermudah akses informasi, mendukung efisiensi kerja, serta memberikan transparansi kondisi barang melalui fitur statistik.

B. Fitur Sistem

Berikut merupakan fitur-fitur yang terdapat pada Sistem Inventaris Kantor:

1. Tambah Barang

Menambahkan data inventaris baru ke dalam sistem, meliputi ID, nama, kategori, dan jumlah barang.

2. Tampilkan Barang

Menampilkan seluruh data barang yang masih aktif dan belum dihapus.

3. Edit Barang

Mengubah informasi barang seperti nama, kategori, dan jumlah sesuai kebutuhan.

4. Hapus Barang (*Soft Delete*)

Menghapus data barang tanpa menghilangkan data secara permanen, hanya mengubah status barang menjadi “dihapus”.

5. Restore Barang

Mengembalikan data barang yang sebelumnya dihapus agar Kembali aktif.

6. Searching Barang

Mencari data barang berdasarkan ID, nama, atau kategori. Pencarian dilakukan dengan algoritma *Linear Search* untuk nama/kategori dan *Binary Search* untuk ID.

7. Sorting Barang

Menyediakan fitur pengurutan data untuk mempermudah pengguna dalam menampilkan informasi barang sesuai kebutuhan. Pengurutan dilakukan dengan beberapa metode, yaitu berdasarkan ID barang menggunakan algoritma *Bubble Sort*, berdasarkan nama barang menggunakan algoritma *Selection Sort*, serta berdasarkan jumlah barang secara *descending* dari jumlah terbesar ke jumlah terkecil. Dengan adanya fitur ini, data inventaris dapat ditampilkan lebih teratur, sehingga memudahkan proses analisis maupun pencarian informasi.

8. Update status barang

Mengubah status barang menjadi “dipinjam”. Selain itu, sistem juga mencatat jumlah peminjaman melalui counter, sehingga riwayat peminjaman barang dapat dipantau dengan jelas dan akurat.

9. Statistik Inventaris

Menampilkan informasi ringkas mengenai kondisi inventaris. Data yang ditampilkan meliputi jumlah total barang, jumlah barang aktif, serta jumlah barang yang sedang dipinjam.

C. Struktur Data yang Digunakan

Dalam pembuatan Sistem Inventaris Kantor, struktur data utama yang digunakan adalah *array of object*. Struktur data ini dipilih karena sesuai untuk pengelolaan data inventaris dengan jumlah data relatif kecil hingga menengah, sehingga penggunaan array lebih sederhana dan mudah dipahami. Array digunakan sebagai wadah penyimpanan seluruh data inventaris, sedangkan object merepresentasikan setiap barang sebagai satu kesatuan data dengan atribut tertentu. Implementasi dapat dilihat pada kode berikut:

```
static Barang[] data = new Barang[100];
```

Gambar 2.1 Implementasi Kode

Kode tersebut menunjukkan bahwa program menggunakan array bernama `data` dengan tipe `Barang` dan kapasitas maksimal 100 data. Setiap indeks array menyimpan objek dari class `Barang`, sehingga satu elemen array merepresentasikan satu data barang inventaris. Class `Barang` berfungsi sebagai blueprint untuk setiap objek inventaris. Di dalamnya terdapat atribut seperti `id` untuk identitas unik, `nama` untuk nama barang, `kategori` untuk jenis barang, `jumlah` untuk stok, `status` untuk kondisi barang (aktif, dipinjam, dihapus), serta `counter` untuk menghitung jumlah peminjaman. Contoh implementasi:

```
class Barang {  
    int id;  
    String nama;  
    String kategori;  
    int jumlah;  
    String status;  
    int counter;  
}
```

Gambar 2.2 Implementasi Kode

Selain itu, sistem ini juga menerapkan konsep **Object Oriented Programming (OOP)** melalui constructor, contohnya:

```
tambahAwal(id: 1, nama: "Laptop", kategori: "Elektronik", jumlah: 10);
```

Gambar 1.3 Implementasi Kode

Kode tersebut menunjukkan bahwa program membuat sebuah objek barang dengan ID 1, nama Laptop, kategori Elektronik, Jumlah 10. Penggunaan *array of object* memberikan keuntungan berupa data yang lebih terstruktur, mudah diakses, dan mendukung implementasi fitur seperti CRUD, pencarian, pengurutan, serta statistik inventaris. Namun, array juga memiliki keterbatasan, seperti kapasitas tetap dan pencarian yang masih linear. Oleh karena itu, struktur data ini lebih cocok digunakan pada sistem dengan jumlah data terbatas, terutama dalam konteks pembelajaran mahasiswa.

D. Algoritma yang Digunakan

Sistem Inventaris Kantor menggunakan beberapa algoritma, yaitu:

1. Linear Search

Digunakan pada fitur pencarian barang berdasarkan ID, nama, dan kategori. Algoritma ini bekerja dengan memeriksa data satu per satu hingga data ditemukan.

2. Buble Sort

Digunakan untuk mengurutkan data berdasarkan ID dan jumlah barang. Algoritma ini bekerja dengan membandingkan dua data berdekatan lalu menukarnya apabila urutan belum sesuai.

3. Selection Sort

Digunakan untuk mengurutkan nama barang secara alfabet. Algoritma ini bekerja dengan mencari data terkecil terlebih dahulu kemudian menukarnya ke posisi awal.

E. Kebutuhan Perangkat

Dalam pengembangan Sistem Inventaris Kantor, terdapat kebutuhan perangkat lunak dan perangkat keras yang harus dipenuhi agar program dapat berjalan dengan baik.

1. Perangkat Lunak (*Software*)

- Java Development Kit (JDK) sebagai komponen utama untuk menjalankan dan mengompilasi program Java.
- Visual Studi Code sebagai *Integrated Development Environment (IDE)* untuk mempermudah proses penulisan, pengujian, dan debugging kode.
- Sistem Operasi Windows sebagai platform utama yang digunakan untuk menjalankan program.

2. Perangkat Keras (*Hardware*)

- Laptop sebagai perangkat utama untuk pengembangan dan eksekusi program.

BAB III

IMPLEMENTASI

A. Struktur Data Utama

Sistem Inventaris Kantor ini dibangun dengan menggunakan paradigma Pemrograman Berorientasi Objek atau *Object-Oriented Programming* (OOP). Dalam OOP, sebuah objek nyata di dunia kerja harus dianalogikan ke dalam sistem digital melalui sebuah cetak biru yang disebut sebagai kelas (*class*). Pada sistem ini, cetak biru tersebut diimplementasikan melalui class `Barang`. Kelas ini mengonseptualisasikan *Abstract Data Type* (ADT) yang bertugas membungkus (*encapsulate*) berbagai variabel dengan tipe data heterogen (kombinasi antara tipe data primitif dan objek/string) menjadi satu kesatuan entitas data yang utuh.

```
class Barang {
    int id;
    String nama;
    String kategori;
    int jumlah;
    String status;
    int counter;

    Barang(int id, String nama, String kategori, int jumlah) {
        this.id = id;
        this.nama = nama;
        this.kategori = kategori;
        this.jumlah = jumlah;
        this.status = "aktif";
        this.counter = 0;
    }
}
```

Gambar 3.1 Struktur Data Utama (Class Barang)

Berdasarkan potongan kode di atas, Class "Barang" memiliki beberapa atribut penunjang informasi, di antaranya:

- ID

Berfungsi sebagai indeks identifikasi primer (*Primary Key*). Atribut ini wajib bersifat unik untuk memastikan tidak ada dua barang berbeda yang memiliki ID yang sama di dalam sistem.

- **Nama**

Menyimpan informasi tekstual mengenai nama spesifik dari aset kantor yang didata (misalnya: "Laptop", "Mouse").

- **Kategori**

Berfungsi sebagai pengelompokan logis aset untuk mempermudah klasifikasi manajemen barang, seperti kategori Elektronik, Furniture, atau ATK.

- **Jumlah**

Menyimpan data kuantitas atau jumlah fisik barang yang tersedia dan tersimpan di dalam gudang inventaris.

- **status**

Atribut penanda status operasional barang. Secara bawaan, konstruktor akan mengatur nilai awalnya sebagai "aktif". Nilai ini akan berubah secara dinamis menjadi "dihapus" atau "dipinjam" seiring dengan manipulasi data yang dilakukan oleh pengguna.

- **Counter**

Berfungsi sebagai perekam data historis. Atribut ini akan menghitung secara akumulatif berapa kali suatu barang telah dipinjam oleh staf kantor.

B. Inisialisasi Dataset Awal

Struktur data penyimpanan utama yang dipilih dalam program ini adalah **Array of Object**. Array digunakan karena memiliki performa akses indeks yang sangat cepat. Agar program dapat langsung diuji kinerjanya tanpa mengharuskan pengguna melakukan proses input manual yang memakan waktu, sistem dilengkapi dengan metode *seeding data* otomatis melalui fungsi `initData()`.

```

21 public class InventarisKantor {
22     static Barang[] data = new Barang[100];
23     static int size = 0;
24     static Scanner input = new Scanner(System.in);
25
26     Run|Debug
27     public static void main(String[] args) {
28         initData();
29
30         int pilihan;
31         do {

```

Gambar 3.2 Deklarasi Array of Object dan Pointer Size.

Ketika fungsi `main()` dijalankan pertama kali, metode `initData()` akan langsung dieksekusi. Di dalam fungsi ini, terdapat 30 baris pemanggilan fungsi `tambahAwal()` yang memasukkan dataset barang secara bertahap. Fungsi `tambahAwal()` bekerja dengan memasukkan objek barang baru ke dalam array sesuai indeks pointer saat itu (`data[size++] = new Barang(...)`). Penggunaan operator *post-increment* (`size++`) memastikan bahwa data dimasukkan ke indeks saat ini terlebih dahulu, baru kemudian nilai variabel `size` bertambah 1 secara otomatis. Proses ini terus berulang hingga indeks ke-29, sehingga saat program siap digunakan, nilai `size` telah bernilai 30.

```

66 // ===== DATA AWAL =====
67 static void initData() {
68     tambahAwal(id: 1, nama: "Laptop", kategori: "Elektronik", jumlah: 10);
69     tambahAwal(id: 2, nama: "Mouse", kategori: "Elektronik", jumlah: 25);
70     tambahAwal(id: 3, nama: "Keyboard", kategori: "Elektronik", jumlah: 20);
71     tambahAwal(id: 4, nama: "Printer", kategori: "Elektronik", jumlah: 5);
72     tambahAwal(id: 5, nama: "Scanner", kategori: "Elektronik", jumlah: 4);
73     tambahAwal(id: 6, nama: "Meja", kategori: "Furniture", jumlah: 15);
74     tambahAwal(id: 7, nama: "Kursi", kategori: "Furniture", jumlah: 30);
75     tambahAwal(id: 8, nama: "Lemari", kategori: "Furniture", jumlah: 10);
76     tambahAwal(id: 9, nama: "AC", kategori: "Elektronik", jumlah: 6);
77     tambahAwal(id: 10, nama: "Proyektor", kategori: "Elektronik", jumlah: 3);
78     tambahAwal(id: 11, nama: "Whiteboard", kategori: "ATK", jumlah: 7);
79     tambahAwal(id: 12, nama: "Spidol", kategori: "ATK", jumlah: 50);
80     tambahAwal(id: 13, nama: "Penghapus", kategori: "ATK", jumlah: 40);
81     tambahAwal(id: 14, nama: "Buku", kategori: "ATK", jumlah: 100);
82     tambahAwal(id: 15, nama: "Pulpen", kategori: "ATK", jumlah: 200);
83     tambahAwal(id: 16, nama: "Stapler", kategori: "ATK", jumlah: 20);
84     tambahAwal(id: 17, nama: "Kertas A4", kategori: "ATK", jumlah: 500);
85     tambahAwal(id: 18, nama: "Map", kategori: "ATK", jumlah: 150);
86     tambahAwal(id: 19, nama: "Rak Buku", kategori: "Furniture", jumlah: 8);
87     tambahAwal(id: 20, nama: "Jam", kategori: "Elektronik", jumlah: 12);
88     tambahAwal(id: 21, nama: "Dispenser", kategori: "Elektronik", jumlah: 5);
89     tambahAwal(id: 22, nama: "Galon", kategori: "Konsumsi", jumlah: 25);
90     tambahAwal(id: 23, nama: "Kipas", kategori: "Elektronik", jumlah: 9);
91     tambahAwal(id: 24, nama: "Telepon", kategori: "Elektronik", jumlah: 6);
92     tambahAwal(id: 25, nama: "Router", kategori: "Elektronik", jumlah: 4);
93     tambahAwal(id: 26, nama: "Flashdisk", kategori: "Elektronik", jumlah: 30);
94     tambahAwal(id: 27, nama: "Harddisk", kategori: "Elektronik", jumlah: 10);
95     tambahAwal(id: 28, nama: "Server", kategori: "Elektronik", jumlah: 2);
96     tambahAwal(id: 29, nama: "Kabel LAN", kategori: "Elektronik", jumlah: 60);
97     tambahAwal(id: 30, nama: "Switch", kategori: "Elektronik", jumlah: 3);
98 }

```

Gambar 3.3 Fungsi Pengisian Dataset Otomatis (*Data Seeding*).

C. Mekanisme CRUD dan Logika Soft Delete

Bagian ini mengatur bagaimana data fungsi manipulasi utama pada sistem inventaris diimplementasikan, meliputi proses penambahan data (*Create*), menampilkan data (*Read*), serta pembaruan data (*Update*). Salah satu aspek krusial yang diterapkan pada sub-bab ini adalah penggunaan metode *Soft Delete* untuk menggantikan mekanisme penghapusan data konvensional atau permanen (*Hard Delete*).

Pada proses penambahan barang melalui fungsi `tambahBarang()`, sistem mengintegrasikan fungsi validasi eksternal bernama `cekID()`. Fungsi `cekID()` bekerja dengan menerima parameter ID inputan pengguna, lalu melakukan iterasi menggunakan perulangan *for* dari indeks 0 hingga `size - 1` untuk memeriksa apakah ID tersebut sudah terdaftar di dalam array. Jika ditemukan kecocokan data, fungsi langsung mengembalikan nilai boolean `true` dan sistem akan membatalkan seluruh rangkaian proses input dengan menampilkan pesan error "ID sudah ada!". Validasi ini sangat vital untuk menjaga integritas data agar tidak terjadi ambiguitas atau duplikasi kunci primer (*Primary Key*).

```
// ===== CRUD =====
static void tambahBarang() {
    if (size >= data.length) {
        System.out.println(x: "Data penuh!");
        return;
    }

    System.out.print(s: "ID: ");
    int id = input.nextInt();

    if (cekID(id)) {
        System.out.println(x: "ID sudah ada!");
        return;
    }

    input.nextLine();
    System.out.print(s: "Nama: ");
    String nama = input.nextLine();

    System.out.print(s: "Kategori: ");
    String kategori = input.nextLine();

    System.out.print(s: "Jumlah: ");
    int jumlah = input.nextInt();

    data[size++] = new Barang(id, nama, kategori, jumlah);
    System.out.println(x: "Barang berhasil ditambahkan!");
}
```

Gambar 3.4 Implementasi Fungsi Tambah Barang dan Validasi ID Unik.

Selanjutnya, pada modul penghapusan data, fungsi `hapusBarang()` menerapkan pendekatan logis *Soft Delete*. Melalui metode ini, ketika operator memilih untuk menghapus barang berdasarkan ID tertentu, sistem tidak akan benar-benar menghapus objek dari memori array ataupun melakukan pergeseran indeks yang dapat memakan waktu komputasi.

Program hanya melakukan pencarian linear untuk menemukan indeks barang tersebut, kemudian mengubah nilai dari atribut variabel `data[i].status` menjadi string "dihapus". Implikasinya, objek barang tersebut tetap aman berada di dalam struktur data internal array, namun secara visual akan disembunyikan dari pengguna. Hal tersebut dibuktikan pada logika fungsi `tampilkanBarang()`, di mana terdapat sebuah kondisi penyaring berupa `if (!data[i].status.equals("dihapus"))` sebelum mencetak baris data ke terminal.

```
static void tampilkanBarang() {
    tampilHeader();
    for (int i = 0; i < size; i++) {
        if (!data[i].status.equals(anObject: "dihapus")) {
            tampil(data[i]);
        }
    }
}

static void editBarang() {
    System.out.print(s: "Masukkan ID: ");
    int id = input.nextInt();
    input.nextLine();

    for (int i = 0; i < size; i++) {
        if (data[i].id == id && !data[i].status.equals(anObject: "dihapus")) {
            System.out.print(s: "Nama baru: ");
            data[i].nama = input.nextLine();

            System.out.print(s: "Kategori baru: ");
            data[i].kategori = input.nextLine();

            System.out.print(s: "Jumlah baru: ");
            data[i].jumlah = input.nextInt();

            System.out.println(x: "Berhasil diupdate!");
            return;
        }
    }
    System.out.println(x: "Barang tidak ditemukan!");
}
```

Gambar 3.5 Logika Penyaringan (*Filtering*) Data Aktif.

```

static void hapusBarang() {
    System.out.print(s: "Masukkan ID: ");
    int id = input.nextInt();

    for (int i = 0; i < size; i++) {
        if (data[i].id == id && !data[i].status.equals(anObject: "dihapus")) {
            data[i].status = "dihapus";
            System.out.println(x: "Berhasil dihapus!");
            return;
        }
    }
    System.out.println(x: "Barang tidak ditemukan!");
}

static void restoreBarang() {
    System.out.print(s: "Masukkan ID: ");
    int id = input.nextInt();

    for (int i = 0; i < size; i++) {
        if (data[i].id == id && data[i].status.equals(anObject: "dihapus")) {
            data[i].status = "aktif";
            System.out.println(x: "Berhasil direstore!");
            return;
        }
    }
    System.out.println(x: "Data tidak ditemukan!");
}

```

Gambar 3.6 Implementasi Fungsi *Soft Delete* dan *Restore* Barang.

Penerapan *Soft Delete* ini memberikan keuntungan besar dalam sistem administrasi inventaris riil, karena jika terjadi kesalahan manusia (*human error*) dalam menghapus data, administrator dapat dengan mudah memulihkan kembali objek tersebut melalui fungsi `restoreBarang()` yang bekerja dengan cara mengembalikan status string menjadi "aktif" kembali.

D. Implementasi Algoritma Sorting

Modul pengurutan data dikembangkan untuk memberikan fleksibilitas analisis inventaris bagi pengguna, sehingga manajemen aset dapat memantau data berdasarkan urutan parameter tertentu. Di dalam sistem ini, program mengimplementasikan dua jenis algoritma pengurutan yang berbeda untuk tipe data yang berbeda, yaitu *Bubble Sort* dan *Selection Sort*.

Algoritma pertama adalah `bubbleSortID()`, yang digunakan untuk mengurutkan data barang berdasarkan nilai ID secara berurutan meningkat (*ascending*). Cara kerja algoritma ini menerapkan metode pertukaran berdampingan dengan memanfaatkan struktur perulangan bersasar (*nested loop*). Perulangan luar (*outer loop*) mengontrol jumlah iterasi penuh, sedangkan perulangan dalam (*inner loop*) bertugas membandingkan nilai indeks `data[j].id` dengan indeks di sebelahnyai yaitu `data[j + 1].id`. Jika elemen kiri lebih besar dari elemen kanan, program memanggil fungsi pembantu `tukar()`. Di dalam fungsi

tukar(), sebuah variabel lokal sementara bernama temp dengan tipe data objek Barang dideklarasikan untuk menampung referensi data agar tidak terhapus selama proses pergeseran tempat di dalam memori array.

Algoritma kedua adalah selectionSortNama(), yang diterapkan untuk mengurutkan nama barang secara alfabetis (*ascending*). Karakteristik *Selection Sort* dipilih karena memiliki keunggulan jumlah operasi pertukaran (*swap*) yang lebih sedikit dan konstan. Logikanya dimulai dengan mengasumsikan indeks iterasi saat ini (i) sebagai posisi nilai terkecil yang disimpan ke dalam variabel penanda min. Perulangan kedua (j) kemudian menyalin sisa array di sebelah kanannya untuk membandingkan string nama menggunakan fungsi bawaan Java .compareToIgnoreCase(). Jika hasil perbandingan bernilai kurang dari nol (< 0), berarti ditemukan nama yang secara alfabetis lebih kecil, sehingga variabel min diperbarui dengan indeks j tersebut. Setelah *inner loop* selesai, program melakukan satu kali pertukaran posisi objek antara indeks i dan indeks min.

Selain itu, sistem juga menyediakan fungsi sortJumlahDesc() untuk mengurutkan barang berdasarkan kuantitas fisik terbanyak ke terkecil (*descending*). Modul ini menggunakan basis logika *Bubble Sort*, namun operator pembandingnya dibalik menjadi `data[j].jumlah < data[j + 1].jumlah` agar nilai yang lebih kecil terdorong ke indeks belakang array.

```
// ===== SORT =====
static void menuSort() {
    System.out.println(x: "\n1. ID (Bubble Sort)");
    System.out.println(x: "2. Nama (Selection Sort)");
    System.out.println(x: "3. Jumlah Desc");
    System.out.print(s: "Pilih: ");
    int pilih = input.nextInt();

    switch (pilih) {
        case 1 -> bubbleSortID();
        case 2 -> selectionSortNama();
        case 3 -> sortJumlahDesc();
    }
    System.out.println(x: "Berhasil diurutkan!");
}

static void bubbleSortID() {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (data[j].id > data[j + 1].id) {
                tukar(j, j + 1);
            }
        }
    }
}
```

Gambar 3.7 Implementasi Fungsi Pengurutan Data (Sorting) dan Fungsi Swap.

```
static void selectionSortNama() {
    for (int i = 0; i < size - 1; i++) {
        int min = i;
        for (int j = i + 1; j < size; j++) {
            if (data[j].nama.compareToIgnoreCase(data[min].nama) < 0) {
                min = j;
            }
        }
        tukar(i, min);
    }
}

static void sortJumlahDesc() {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (data[j].jumlah < data[j + 1].jumlah) {
                tukar(j, j + 1);
            }
        }
    }
}

static void tukar(int i, int j) {
    Barang temp = data[i];
    data[i] = data[j];
    data[j] = temp;
}
```

Gambar 3.8 Implementasi Fungsi Pengurutan Data (Sorting) dan Fungsi Swap.

E. Implementasi Algoritma Searching

Fitur pencarian (*searching*) merupakan modul krusial yang dipanggil pada opsi menu keenam untuk mempermudah pengguna dalam menemukan lokasi data aset kantor secara spesifik tanpa harus menyisir seluruh isi tabel inventaris. Sistem ini mengimplementasikan algoritma **Linear Search** (pencarian sekuensial atau berurutan). Algoritma ini dipilih karena struktur penyimpanan data internal berbasis array yang digunakan berukuran moderat (100 slot), sehingga proses pemindaian satu per satu dari indeks awal hingga indeks aktif ($\text{size} - 1$) masih sangat efisien, responsif, dan hemat memori.

Di dalam pengembangannya, modul pencarian ini diekspansi menjadi tiga fungsi spesifik berdasarkan atribut orientasinya melalui metode `menuSearch()`:

- **Pencarian Nama (`cariNama()`):** Untuk meningkatkan fleksibilitas dan pengalaman pengguna (*User Experience*), pencarian berdasarkan nama dirancang tidak sensitif terhadap huruf kapital (*case-insensitive*) dan mendukung pencarian klausa parsial. Logikanya adalah dengan mengubah input kata kunci user dan nama barang di array menjadi huruf kecil terlebih dahulu memakai fungsi `.toLowerCase()`. Kemudian, dengan memanfaatkan fungsi bawaan `.contains()`, sistem akan mencocokkan apakah string nama mengandung potongan kata kunci tersebut. Sebagai contoh, jika pengguna

mengetik kata kunci "top", maka objek bernama "Laptop" akan tetap terjaring dan ditampilkan ke layar.

- **Pencarian Kategori (cariKategori()):** Berbeda dengan pencarian nama, fungsi ini mencocokkan kategori secara absolut menggunakan fungsi `.equals()` setelah disamakan format hurufnya. Fungsi ini menyalisir seluruh array untuk menampilkan kelompok barang yang memiliki rumpun sejenis (misalnya menampilkan semua barang berkategori "Elektronik" atau "ATK").
- **Pencarian ID (cariID()):** Karena atribut ID bertindak sebagai identitas unik (*Primary Key*), proses pencarian ID dibuat sangat ketat dengan operator kesamaan `==`. Segera setelah program menemukan objek dengan ID yang cocok dan berstatus aktif, sistem akan langsung mencetak data tersebut dan mengeksekusi perintah `return`. Instruksi `return` ini secara instan mematahkan perulangan *for* dan keluar dari fungsi tanpa perlu memeriksa sisa indeks array di bawahnya, sehingga menghemat siklus kerja prosesor komputer.

Pada ketiga fungsi pencarian di atas, program juga menyisipkan kondisi `!data[i].status.equals("dihapus")`. Hal ini memastikan bahwa barang yang sudah dikenakan operasi *soft delete* tidak akan ikut terambil atau muncul dalam hasil pencarian pengguna.

```
// ===== SEARCH =====
static void menuSearch() {
    System.out.println(x: "\n1. Nama");
    System.out.println(x: "2. ID");
    System.out.println(x: "3. Kategori");
    System.out.print(s: "Pilih: ");
    int pilih = input.nextInt();
    input.nextLine();

    switch (pilih) {
        case 1 -> cariNama();
        case 2 -> cariID();
        case 3 -> cariKategori();
    }
}

static void cariNama() {
    System.out.print(s: "Nama: ");
    String nama = input.nextLine().toLowerCase();

    tampilHeader();
    for (int i = 0; i < size; i++) {
        if (data[i].nama.toLowerCase().contains(nama) && !data[i].status.equals(anObject: "dihapus")) {
            tampil(data[i]);
        }
    }
}
```

Gambar 3.9 Implementasi Fungsi Search


```

static void cariKategori() {
    System.out.print(s: "Kategori: ");
    String kategori = input.nextLine().toLowerCase();

    tampilHeader();
    for (int i = 0; i < size; i++) {
        if (data[i].kategori.toLowerCase().equals(kategori) && !data[i].status.equals(anObject: "dihapus")) {
            tampil(data[i]);
        }
    }
}

static void cariID() {
    System.out.print(s: "ID: ");
    int id = input.nextInt();

    for (int i = 0; i < size; i++) {
        if (data[i].id == id && !data[i].status.equals(anObject: "dihapus")) {
            tampilHeader();
            tampil(data[i]);
            return;
        }
    }
    System.out.println(x: "Tidak ditemukan!");
}

```

Gambar 3.10 Implementasi Fungsi Search.

BAB IV

ANALISIS KOMPLEKSITAS

A. Pendahuluan Analisis Kompleksitas

Analisis kompleksitas merupakan metode untuk mengukur efisiensi suatu algoritma dari sisi **waktu eksekusi (Time Complexity)** dan **penggunaan memori (Space Complexity)**. Dalam laporan ini, analisis dilakukan terhadap seluruh fungsi yang terdapat pada program Sistem Inventaris Kantor berbasis Java.

Notasi yang digunakan adalah **Big-O Notation**, yaitu notasi asimptotik yang menggambarkan pertumbuhan waktu atau ruang suatu algoritma terhadap ukuran input **n** (jumlah data dalam array). Analisis dilakukan pada tiga skenario, yaitu:

- Best Case (Ω): kondisi terbaik, data ditemukan atau kondisi terpenuhi di iterasi pertama.
- Average Case (Θ): kondisi rata-rata, representasi umum performa algoritma.
- Worst Case (O): kondisi terburuk, data tidak ditemukan atau harus menelusuri seluruh array.

B. Analisis Time Complexity

Tabel 4.1 Rekapitulasi Time Complexity Seluruh Fungsi

No.	Fungsi / Operasi	Best Case	Worst Case	Average Case
1	tambahBarang()	$O(n)$	$O(n)$	$O(n)$
2	cekID()	$O(1)$	$O(n)$	$O(n)$
3	tampilkanBarang()	$O(n)$	$O(n)$	$O(n)$
4	editBarang()	$O(1)$	$O(n)$	$O(n)$

5	hapusBarang() – Soft Delete	$O(1)$	$O(n)$	$O(n)$
6	restoreBarang()	$O(1)$	$O(n)$	$O(n)$
7	cariNama()	$O(1)$	$O(n)$	$O(n)$
8	cariID()	$O(1)$	$O(n)$	$O(n)$
9	cariKategori()	$O(1)$	$O(n)$	$O(n)$
10	bubbleSortID()	$O(n^2)$	$O(n^2)$	$O(n^2)$
11	selectionSortNama()	$O(n^2)$	$O(n^2)$	$O(n^2)$
12	sortJumlahDesc()	$O(n^2)$	$O(n^2)$	$O(n^2)$
13	updateStatus()	$O(1)$	$O(n)$	$O(n)$
14	statistik()	$O(n)$	$O(n)$	$O(n)$
15	initData()	$O(n)$	$O(n)$	$O(n)$

1. Operasi CRUD (Create, Read, Update, Delete)

- tambahBarang() dan cekID()

Fungsi **tambahBarang()** memanggil **cekID()** untuk memvalidasi keunikan ID sebelum menyimpan data. Fungsi **cekID()** melakukan penelusuran linier pada array **data[0..size-1]** sebanyak satu kali untuk memeriksa apakah ID sudah ada.

Best Case: $O(1)$ — ID ditemukan duplikat di elemen pertama, iterasi langsung berhenti.

Worst Case: $O(n)$ — ID unik, seluruh array ditelusuri tanpa menemukan duplikat.

Karena **tambahBarang()** bergantung pada **cekID()**, kompleksitas keseluruhannya adalah $O(n)$.

- tampilkanBarang()

Fungsi ini melakukan satu loop penuh dari indeks 0 sampai **size-1**, mencetak setiap barang yang statusnya bukan 'dihapus'. Tidak ada early exit, sehingga kompleksitas selalu **$O(n)$** pada semua skenario.

- `editBarang()`, `hapusBarang()`, `restoreBarang()`, `updateStatus()`

Keempat fungsi ini memiliki pola yang sama: mencari ID target dengan sequential search, lalu melakukan operasi pada elemen yang ditemukan.

Best Case: $O(1)$ — ID yang dicari berada di indeks pertama array.

Worst Case: $O(n)$ — ID berada di indeks terakhir atau tidak ditemukan sama sekali.

2. Operasi Pencarian (Search)

Program mengimplementasikan algoritma **Sequential Search (Linear Search)** pada ketiga fungsi pencarian: `cariNama()`, `cariID()`, dan `cariKategori()`. Seluruhnya menelusuri array secara berurutan dari awal.

Tabel 4.2 Analisis Kompleksitas Operasi Pencarian

Algoritma	Best Case	Worst Case	Keterangan
Sequential Search (Nama)	$O(1)$	$O(n)$	Data ditemukan di indeks pertama / terakhir
Sequential Search (ID)	$O(1)$	$O(n)$	Data ditemukan di indeks pertama / terakhir
Sequential Search (Kategori)	$O(1)$	$O(n)$	Mencocokkan seluruh array secara linier

Catatan: `cariNama()` dan `cariKategori()` tidak melakukan early exit (tidak ada return setelah menemukan data), sehingga tetap menelusuri seluruh array meskipun data sudah ditemukan. Hal ini menyebabkan kompleksitas sesungguhnya selalu $O(n)$. Pengecualian hanya pada `cariID()` yang menggunakan return setelah data pertama ditemukan.

3. Operasi Pengurutan (Sort)

Program mengimplementasikan dua algoritma pengurutan, yaitu **Bubble Sort** dan **Selection Sort**. Kedua algoritma ini menggunakan nested loop sehingga tergolong dalam kelas $O(n^2)$.

Tabel 4.3 Analisis Kompleksitas Operasi Pengurutan

Algoritma	Best Case	Worst Case	Keterangan
Bubble Sort (ID)	$O(n^2)$	$O(n^2)$	Membandingkan elemen berdekatan secara berulang
Selection Sort (Nama)	$O(n^2)$	$O(n^2)$	Memilih minimum lalu tukar ke posisi yang benar
Bubble Sort Desc (Jumlah)	$O(n^2)$	$O(n^2)$	Sama seperti Bubble Sort, arah descending

- bubbleSortID() – Bubble Sort

Bubble Sort bekerja dengan membandingkan dua elemen berdekatan secara berulang dan menukarnya jika urutan tidak sesuai. Kode menggunakan dua loop bersarang:

Loop luar berjalan sebanyak (size - 1) kali $\rightarrow O(n)$

Loop dalam berjalan sebanyak (size - i - 1) kali $\rightarrow$ rata-rata $O(n)$

Total: $O(n) \times O(n) = O(n^2)$

Implementasi ini **tidak memiliki optimasi early-exit**, sehingga best case tetap $O(n^2)$ meskipun array sudah terurut.

- selectionSortNama() – Selection Sort

Selection Sort bekerja dengan mencari nilai minimum dari sisa array yang belum terurut, kemudian menukarnya ke posisi yang benar.

Loop luar berjalan sebanyak (size - 1) kali $\rightarrow O(n)$

Loop dalam mencari elemen minimum di sisa array $\rightarrow O(n)$

Total: $O(n^2)$ untuk semua kasus (best, average, worst)

Perbandingan nama menggunakan **compareToIgnoreCase()** yang memiliki kompleksitas $O(k)$ di mana k adalah panjang string. Namun, karena k dianggap konstan dalam konteks ini, kompleksitas keseluruhan tetap $O(n^2)$.

4. Fungsi Statistik

Fungsi **statistik()** melakukan satu kali penelusuran linear pada seluruh array untuk menghitung jumlah barang aktif, dipinjam, dan total. Hanya menggunakan tiga variabel counter sehingga kompleksitas waktu adalah **O(n)** dan kompleksitas ruang adalah **O(1)**.

C. Analisis Space Complexity

Space Complexity mengukur jumlah memori yang digunakan suatu algoritma selain memori untuk menyimpan input utama. Berikut analisis penggunaan ruang memori pada program:

Tabel 4.4 Analisis Space Complexity Program

No.	Fungsi / Struktur	Space Complexity	Keterangan
1	Array data[100]	O(n)	Menyimpan maks. 100 objek Barang
2	Objek Barang	O(1)	6 field per objek (konstan)
3	bubbleSortID()	O(1)	In-place, hanya 1 variabel temp
4	selectionSortNama()	O(1)	In-place, hanya variabel indeks min
5	sortJumlahDesc()	O(1)	In-place, hanya 1 variabel temp
6	cariNama() / cariID() / cariKategori()	O(1)	Tidak alokasi memori tambahan
7	statistik()	O(1)	Hanya 3 variabel counter
8	Scanner input	O(1)	Satu instance global

Secara keseluruhan, program menggunakan **O(n) space** untuk struktur data utama (array data[100]) dan **O(1) auxiliary space** untuk semua operasi. Ini berarti program sangat efisien dalam

penggunaan memori karena semua algoritma sorting dan searching berjalan **in-place** tanpa memerlukan alokasi memori tambahan yang bergantung pada ukuran input.

D. Perbandingan dan Evaluasi

1. Kekuatan Implementasi

- Semua algoritma sorting menggunakan pendekatan in-place sehingga space complexity tetap $O(1)$.
- Soft delete (status = 'dihapus') menghindari operasi penghapusan fisik data yang mahal, menjaga integritas indeks array.
- Fungsi cariID() memiliki early exit yang mengoptimalkan pencarian berdasarkan ID unik.
- Inisialisasi data awal (initData) menggunakan pemanggilan berulang tambahAwal() dengan kompleksitas $O(n)$ total — efisien untuk data statis.

2. Kelemahan dan Saran Perbaikan

- Bubble Sort dan Selection Sort memiliki kompleksitas $O(n^2)$. Untuk dataset yang besar, disarankan menggunakan algoritma yang lebih efisien seperti Merge Sort $O(n \log n)$ atau Quick Sort $O(n \log n)$.
- cariNama() dan cariKategori() tidak memiliki early exit; meskipun data sudah ditemukan, loop tetap berjalan hingga akhir. Penambahan kondisi break atau flag dapat meningkatkan performa pada best case.
- Penggunaan array statis berukuran tetap (data[100]) membatasi skalabilitas. Penggunaan ArrayList atau struktur data dinamis dapat mengatasi keterbatasan ini.
- Tidak ada implementasi Binary Search, padahal setelah data diurutkan, Binary Search dapat meningkatkan pencarian dari $O(n)$ menjadi $O(\log n)$.

E. Kesimpulan Analisis Kompleksitas

Berdasarkan analisis yang telah dilakukan, program Sistem Inventaris Kantor secara umum beroperasi dengan efisiensi yang memadai untuk skala data yang digunakan (maksimum 100

barang). Dominasi kompleksitas $O(n)$ pada operasi CRUD dan pencarian menunjukkan performa linier yang dapat diterima. Namun, penggunaan algoritma $O(n^2)$ pada operasi pengurutan perlu menjadi perhatian apabila program dikembangkan untuk menangani dataset yang lebih besar.

Program ini juga sangat efisien dalam penggunaan memori dengan auxiliary space $O(1)$ pada seluruh operasi, menjadikannya cocok untuk dijalankan pada lingkungan dengan keterbatasan memori. Secara keseluruhan, desain program sudah cukup baik sebagai implementasi sistem inventaris sederhana berbasis array.

BAB V

PENGUJIAN

A. Tujuan Pengujian

Pengujian sistem dilakukan untuk memastikan bahwa seluruh fitur pada Sistem Inventaris Kantor dapat berjalan dengan baik sesuai dengan fungsi dan kebutuhan pengguna. Proses pengujian dilakukan pada setiap fitur utama program, mulai dari pengelolaan data barang, pencarian data, pengurutan data, hingga proses menampilkan statistik inventaris.

Selain itu, pengujian bertujuan untuk mengetahui apakah sistem mampu menerima input, memproses data, dan menghasilkan output yang sesuai dengan yang diharapkan. Pengujian juga dilakukan untuk memastikan tidak terdapat kesalahan pencarian, maupun pengurutan data inventaris.

Melalui proses pengujian ini, dapat diketahui Tingkat keberhasilan implementasi struktur data array, konsep Object Oriented Programming (OOP), serta algoritma *searching* dan *sorting* yang digunakan pada program. Hasil pengujian diharapkan dapat menunjukkan bahwa sistem inventaris yang dibuat telah berjalan secara efektif, terstruktur, dan mampu membantu proses pengelolaan inventaris kantor dengan lebih baik.

B. Skenario Pengujian

Pengujian pada Sistem Inventaris Kantor dilakukan dengan mencoba seluruh fitur yang tersedia pada program menggunakan berbagai kondisi input data. Setiap pengujian bertujuan untuk memastikan bahwa sistem mampu menjalankan fungsi sesuai dengan yang diharapkan serta menghasilkan output yang benar.

Proses pengujian dilakukan secara bertahap mulai dari pengelolaan data inventaris, proses pencarian dan pengurutan data, hingga pengujian statistik inventaris. Selain itu, pengujian juga dilakukan untuk memastikan sistem dapat menangani kondisi tertentu seperti data tidak ditemukan, ID yang sudah digunakan, maupun proses *restore* data yang telah dihapus.

Berikut merupakan skenario pengujian yang dilakukan pada sistem:

1. Pengujian Menjalankan Program

```
=== SISTEM INVENTARIS KANTOR ===  
1. Tambah Barang  
2. Tampilkan Barang  
3. Edit Barang  
4. Hapus Barang (Soft Delete)  
5. Restore Barang  
6. Cari Barang  
7. Urutkan Barang  
8. Update Status  
9. Statistik  
10. Keluar  
Pilih:
```

Gambar 5.1 Pengujian Menjalankan Program

Pada pengujian ini, program berhasil dijalankan tanpa mengalami error. Sistem mampu menampilkan menu utama yang berisi seluruh fitur inventaris kantor sehingga pengguna dapat memilih menu sesuai kebutuhan.

2. Pengujian Tambah Barang

```
Pilih: 1  
ID: 031  
Nama: binder garis  
Kategori: alat tulis kantor  
Jumlah: 50  
Barang berhasil ditambahkan!  
  
=== SISTEM INVENTARIS KANTOR ===  
1. Tambah Barang  
2. Tampilkan Barang  
3. Edit Barang  
4. Hapus Barang (Soft Delete)  
5. Restore Barang  
6. Cari Barang  
7. Urutkan Barang  
8. Update Status  
9. Statistik  
10. Keluar
```

gambar 5.2 Pengujian Menambah Barang

Hasil pengujian menunjukkan bahwa sistem berhasil menambahkan data barang baru ke dalam array inventaris. Data yang dimasukkan adalah binder garis, dengan ID 031, kategori alat tulis kantor, dan jumlah 50. Data tersebut tersimpan dengan baik dan dapat ditampilkan kembali pada menu daftar barang.

3. Pengujian ID yang Sudah Ada

```
Pilih: 1
ID: 20
ID sudah ada!

=== SISTEM INVENTARIS KANTOR ===
1. Tambah Barang
2. Tampilkan Barang
3. Edit Barang
4. Hapus Barang (Soft Delete)
5. Restore Barang
6. Cari Barang
7. Urutkan Barang
8. Update Status
9. Statistik
10. Keluar
```

Gambar 5.3 Pengujian ID yang Sudah Ada

Saat pengguna memasukkan ID yang telah digunakan sebelumnya, sistem berhasil melakukan validasi dan menampilkan pesan “ID sudah ada”. Hal ini menunjukkan bahwa sistem mampu mencegah terjadinya data ganda.

4. Pengujian Tampilkan Barang

Pilih: 2

ID	Nama	Kategori	Jumlah	Status	Dipinjam
1	Laptop	Elektronik	10	aktif	0
2	Mouse	Elektronik	25	aktif	0
3	Keyboard	Elektronik	20	aktif	0
4	Printer	Elektronik	5	aktif	0
5	Scanner	Elektronik	4	aktif	0
6	Meja	Furniture	15	aktif	0
7	Kursi	Furniture	30	aktif	0
8	Lemari	Furniture	10	aktif	0
9	AC	Elektronik	6	aktif	0
10	Proyektor	Elektronik	3	aktif	0
11	Whiteboard	ATK	7	aktif	0
12	Spidol	ATK	50	aktif	0
13	Penghapus	ATK	40	aktif	0
14	Buku	ATK	100	aktif	0
15	Pulpen	ATK	200	aktif	0
16	Stapler	ATK	20	aktif	0
17	Kertas A4	ATK	500	aktif	0
18	Map	ATK	150	aktif	0
19	Rak Buku	Furniture	8	aktif	0
20	Jam	Elektronik	12	aktif	0
21	Dispenser	Elektronik	5	aktif	0
22	Galon	Konsumsi	25	aktif	0
23	Kipas	Elektronik	9	aktif	0
24	Telepon	Elektronik	6	aktif	0
25	Router	Elektronik	4	aktif	0
26	Flashdisk	Elektronik	30	aktif	0
27	Harddisk	Elektronik	10	aktif	0
28	Server	Elektronik	2	aktif	0
29	Kabel LAN	Elektronik	60	aktif	0
30	Switch	Elektronik	3	aktif	0

Gambar 5.4 Pengujian Menampilkan Barang

Sistem berhasil menampilkan seluruh data inventaris yang masih aktif dalam bentuk tabel. Data yang ditampilkan meliputi ID, nama barang, kategori, jumlah, status, dan jumlah peminjaman barang.

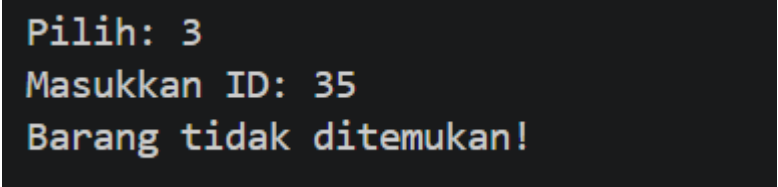
5. Pengujian Edit Barang

```
Pilih: 3
Masukkan ID: 15
Nama baru: pulpen merah
Kategori baru: alat tulis kantor
Jumlah baru: 50
Berhasil diupdate!
```

Gambar 5.5 Pengujian Mengedit Barang

Pada pengujian ini, sistem berhasil memperbarui data barang berdasarkan ID yang dipilih pengguna. Perubahan data dapat langsung terlihat setelah proses edit selesai dilakukan.

6. Pengujian Edit Data yang Tidak Tersedia

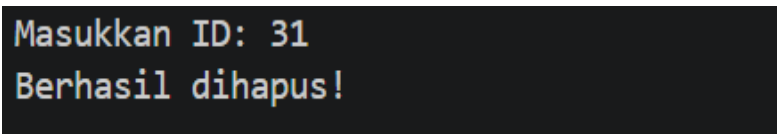


```
Pilih: 3
Masukkan ID: 35
Barang tidak ditemukan!
```

Gambar 5.6 Pengujian Mengedit Data yang Tidak Tersedia

Ketika pengguna memasukkan ID barang yang tidak tersedia, sistem menampilkan pesan bahwa barang tidak ditemukan. Hal ini menunjukkan bahwa sistem mampu menangani kesalahan input dengan baik.

7. Pengujian Hapus Barang (*Soft Delete*)

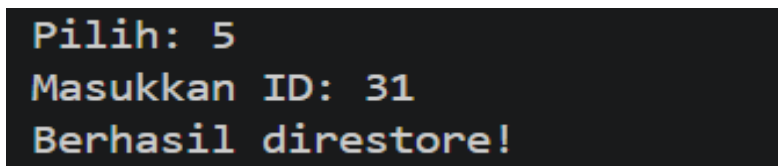


```
Masukkan ID: 31
Berhasil dihapus!
```

Gambar 5.7 Pengujian *Soft Delete*

Hasil pengujian menunjukkan bahwa fitur soft delete berjalan dengan baik. Sistem tidak menghapus data secara permanen, melainkan hanya mengubah status barang menjadi “dihapus”.

8. Pengujian Restore Barang



```
Pilih: 5
Masukkan ID: 31
Berhasil direstore!
```

Gambar 5.8 Pengujian *Restore* Barang

Data barang yang sebelumnya dihapus berhasil dikembalikan menggunakan fitur *restore*. Status barang berubah kembali menjadi “aktif” sehingga data dapat digunakan kembali.

9. Pengujian *Searching* Berdasarkan ID

```
Pilih: 6
1. Nama
2. ID
3. Kategori
Pilih: 2
ID: 24
ID      Nama      Kategori      Jumlah      Status      Dipinjam
-----
24      Telepon    Elektronik    6           aktif       0
```

Gambar 5.9 Pengujian *Searching* Berdasarkan ID

Sistem berhasil menemukan data barang sesuai ID yang dimasukkan pengguna, yaitu 24. Proses pencarian dilakukan menggunakan algoritma *Linear Search* dengan memeriksa data satu per satu.

10. Pengujian *Searching* Berdasarkan Nama

```
Pilih: 6
1. Nama
2. ID
3. Kategori
Pilih: 1
Nama: Dispenser
ID      Nama      Kategori      Jumlah      Status      Dipinjam
-----
21      Dispenser    Elektronik    5           aktif       0
```

Gambar 5.10 Pengujian *Searching* Berdasarkan Nama

Pada pengujian ini, sistem berhasil menampilkan data barang berdasarkan nama yang dimasukkan pengguna yaitu Dispenser. Hal ini mempermudah pengguna dalam mencari barang inventaris.

11. Pengujian *Searching* Berdasarkan Kategori

```
Pilih: 6
1. Nama
2. ID
3. Kategori
Pilih: 3
Kategori: furniture
ID      Nama      Kategori      Jumlah      Status      Dipinjam
-----
6       Meja      Furniture     15          aktif       0
7       Kursi     Furniture     30          aktif       0
8       Lemari    Furniture     10          aktif       0
19      Rak Buku  Furniture     8           aktif       0
```

Gambar 5.11 Pengujian *Searching* Berdasarkan Kategori

Sistem mampu menampilkan seluruh barang sesuai kategori yang dipilih pengguna yaitu furniture. Pengujian ini menunjukkan bahwa fitur pencarian kategori berjalan dengan baik.

12. Pengujian *Searching* Data yang Tidak Ada

```
Pilih: 6

1. Nama
2. ID
3. Kategori
Pilih: 2
ID: 35
Tidak ditemukan!
```

Gambar 5.12 Pengujian *Searching* Data yang Tidak Ada

Ketika data yang dicari tidak tersedia, sistem berhasil menampilkan pesan “Tidak ditemukan”. Hal ini menunjukkan bahwa sistem mampu menangani kondisi pencarian gagal dengan benar.

13. Pengujian *Sorting* Berdasarkan ID

```
Pilih: 7

1. ID (Bubble Sort)
2. Nama (Selection Sort)
3. Jumlah Desc
Pilih: 1
Berhasil diurutkan!
```

Gambar 5.13 Pengujian *Sorting* Berdasarkan ID

Hasil:

1	Laptop	Elektronik	10	aktif	0
2	Mouse	Elektronik	25	aktif	0
3	Keyboard	Elektronik	20	aktif	0
4	Printer	Elektronik	5	aktif	0
5	Scanner	Elektronik	4	aktif	0
6	Meja	Furniture	15	aktif	0
7	Kursi	Furniture	30	aktif	0
8	Lemari	Furniture	10	aktif	0
9	AC	Elektronik	6	aktif	0
10	Proyektor	Elektronik	3	aktif	0
11	Whiteboard	ATK	7	aktif	0
12	Spidol	ATK	50	aktif	0
13	Penghapus	ATK	40	aktif	0
14	Buku	ATK	100	aktif	0
15	pulpen merah	alat tulis kantor	50	aktif	0
16	Stapler	ATK	20	aktif	0
17	Kertas A4	ATK	500	aktif	0
18	Map	ATK	150	aktif	0
19	Rak Buku	Furniture	8	aktif	0
20	Jam	Elektronik	12	aktif	0
21	Dispenser	Elektronik	5	aktif	0
22	Galon	Konsumsi	25	aktif	0
23	Kipas	Elektronik	9	aktif	0
24	Telepon	Elektronik	6	aktif	0
25	Router	Elektronik	4	aktif	0
26	Flashdisk	Elektronik	30	aktif	0
27	Harddisk	Elektronik	10	aktif	0
28	Server	Elektronik	2	aktif	0
29	Kabel LAN	Elektronik	60	aktif	0
30	Switch	Elektronik	3	aktif	0
31	binder garis	alat tulis kantor	50	aktif	0

Hasil pengujian menunjukkan bahwa data barang berhasil diurutkan berdasarkan ID menggunakan algoritma *Bubble Sort*. Urutan data berubah dari ID terkecil ke terbesar.

14. Pengujian *Sorting* Berdasarkan Nama

```
Pilih: 7

1. ID (Bubble Sort)
2. Nama (Selection Sort)
3. Jumlah Desc
Pilih: 2
Berhasil diurutkan!
```

Gambar 5.14 Pengujian *Sorting* Berdasarkan Nama

Hasil:

Pilih: 2

ID	Nama	Kategori	Jumlah	Status	Dipinjam
9	AC	Elektronik	6	aktif	0
31	binder garis	alat tulis kantor	50	aktif	0
14	Buku	ATK	100	aktif	0
21	Dispenser	Elektronik	5	aktif	0
26	Flashdisk	Elektronik	30	aktif	0
22	Galon	Konsumsi	25	aktif	0
27	Harddisk	Elektronik	10	aktif	0
20	Jam	Elektronik	12	aktif	0
29	Kabel LAN	Elektronik	60	aktif	0
17	Kertas A4	ATK	500	aktif	0
3	Keyboard	Elektronik	20	aktif	0
23	Kipas	Elektronik	9	aktif	0
7	Kursi	Furniture	30	aktif	0
1	Laptop	Elektronik	10	aktif	0
8	Lemari	Furniture	10	aktif	0
18	Map	ATK	150	aktif	0
6	Meja	Furniture	15	aktif	0
2	Mouse	Elektronik	25	aktif	0
13	Penghapus	ATK	40	aktif	0
4	Printer	Elektronik	5	aktif	0
10	Proyektor	Elektronik	3	aktif	0
15	pulpen merah	alat tulis kantor	50	aktif	0
19	Rak Buku	Furniture	8	aktif	0
25	Router	Elektronik	4	aktif	0
5	Scanner	Elektronik	4	aktif	0
28	Server	Elektronik	2	aktif	0
12	Spidol	ATK	50	aktif	0
16	Stapler	ATK	20	aktif	0
30	Switch	Elektronik	3	aktif	0

Pada pengujian ini, sistem berhasil mengurutkan nama barang secara alfabetis menggunakan algoritma *Selection Sort*. Data tampil lebih rapi dan mudah dicari.

15. Pengujian *Sorting* Berdasarkan Jumlah

```
Pilih: 7

1. ID (Bubble Sort)
2. Nama (Selection Sort)
3. Jumlah Desc
Pilih: 3
Berhasil diurutkan!
```

Gambar 5.15 Pengujian *Sorting* Berdasarkan Jumlah

Hasil:

Pilih: 2					
ID	Nama	Kategori	Jumlah	Status	Dipinjam

17	Kertas A4	ATK	500	aktif	0
18	Map	ATK	150	aktif	0
14	Buku	ATK	100	aktif	0
29	Kabel LAN	Elektronik	60	aktif	0
31	binder garis	alat tulis kantor	50	aktif	0
15	pulpen merah	alat tulis kantor	50	aktif	0
12	Spidol	ATK	50	aktif	0
13	Penghapus	ATK	40	aktif	0
26	Flashdisk	Elektronik	30	aktif	0
7	Kursi	Furniture	30	aktif	0
22	Galon	Konsumsi	25	aktif	0
2	Mouse	Elektronik	25	aktif	0
3	Keyboard	Elektronik	20	aktif	0
16	Stapler	ATK	20	aktif	0
6	Meja	Furniture	15	aktif	0
20	Jam	Elektronik	12	aktif	0
27	Harddisk	Elektronik	10	aktif	0
1	Laptop	Elektronik	10	aktif	0
8	Lemari	Furniture	10	aktif	0
23	Kipas	Elektronik	9	aktif	0
19	Rak Buku	Furniture	8	aktif	0
11	Whiteboard	ATK	7	aktif	0
9	AC	Elektronik	6	aktif	0
24	Telepon	Elektronik	6	aktif	0
21	Dispenser	Elektronik	5	aktif	0
4	Printer	Elektronik	5	aktif	0
25	Router	Elektronik	4	aktif	0
5	Scanner	Elektronik	4	aktif	0
10	Proyektor	Elektronik	3	aktif	0
30	Switch	Elektronik	3	aktif	0

Sistem berhasil mengurutkan data berdasarkan jumlah barang secara descending, yaitu dari jumlah terbesar ke terkecil. Pengujian ini membantu pengguna melihat stok barang terbanyak dengan lebih mudah.

16. Pengujian *Update* Status Barang

```
Pilih: 8  
ID: 12  
Status diperbarui!
```

Gambar 5.16 Pengujian *Update* Status Barang

Hasil:

12	Spidol	ATK	50	dipinjam	1
----	--------	-----	----	----------	---

Fitur update status berhasil mengubah status barang menjadi “dipinjam”. Selain itu, sistem juga berhasil menambahkan jumlah peminjaman pada barang tersebut.

17. Pengujian Statistik Inventaris

```
=== STATISTIK ===  
Total: 31  
Aktif: 30  
Dipinjam: 1
```

Gambar 5.17 Pengujian Statistik Inventaris

Hasil pengujian menunjukkan bahwa sistem mampu menghitung dan menampilkan statistik inventaris dengan benar, seperti total barang, jumlah barang aktif, dan jumlah barang yang sedang dipinjam.

BAB VI

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, analisis kompleksitas, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa Sistem Inventaris Kantor berbasis Java berhasil dibuat dan mampu membantu proses pengelolaan inventaris secara lebih terstruktur, cepat, dan efisien dibandingkan pencatatan manual. Sistem ini dibuat untuk mengatasi permasalahan seperti kesulitan pencarian barang, data yang kurang terorganisir, serta proses pengelolaan inventaris yang memerlukan waktu cukup lama.

Program berhasil mengimplementasikan konsep Object Oriented Programming (OOP), struktur data array, serta algoritma searching dan sorting dalam proses pengolahan data inventaris. Algoritma Linear Search digunakan pada fitur pencarian barang, sedangkan Bubble Sort dan Selection Sort digunakan pada proses pengurutan data. Berdasarkan hasil analisis kompleksitas, sebagian besar operasi memiliki kompleksitas $O(n)$, sedangkan proses sorting memiliki kompleksitas $O(n^2)$, namun masih cukup efektif untuk jumlah data yang digunakan pada sistem.

Selain itu, hasil pengujian menunjukkan bahwa seluruh fitur program seperti tambah data, tampil data, edit data, soft delete, restore data, searching, sorting, update status, dan statistik inventaris dapat berjalan dengan baik sesuai dengan kebutuhan pengguna. Dengan demikian, sistem yang dibuat telah mampu memenuhi fungsi dasar pengelolaan inventaris kantor sekaligus menjadi sarana pembelajaran dalam penerapan struktur data dan algoritma pada studi kasus nyata.